Abon, Benedict Aldous A. CPE311 - CPE22S3 03/29/26 03/29/26 Engr. Neil Barton James Matira

Seatwork 10.1 Case Study: Improving RT-IoT2022 Analysis

Guide Questions

- What is the distribution of the Attack_type classes (normal vs. various attacks), and what percentage of the 123,117 instances does each class comprise?
- How do the categorical features proto (protocol) and service vary across different attack types and normal traffic patterns? Links to an external site.
- What are the mean and standard deviation of flow_duration for each Attack_type, and are differences statistically significant?
- Which continuous features (e.g., fwd_pkts_per_sec, bwd_pkts_per_sec, payload_bytes_per_second) exhibit the highest correlation with specific attack classes?
- How do time-based features like fwd_iat.avg and bwd_iat.avg (mean inter-arrival times) differ between various attack types and normal traffic?
- Which network flag counts (e.g., flow_SYN_flag_count, flow_RST_flag_count, fwd_PSH_flag_count) are most indicative of specific intrusion patterns?

Extract

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("RT_IOT2022.csv")
df.head()

{"type":"dataframe","variable_name":"df"}

print(f"df shape: {df.shape}")

df shape: (71726, 85)

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71726 entries, 0 to 71725
Data columns (total 85 columns):
 #   Column                    Non-Null Count  Dtype
```

```
---   ------                      --------------  -----
 0    no                          71726 non-null  int64
 1    id.orig_p                   71726 non-null  int64
 2    id.resp_p                   71726 non-null  int64
 3    proto                       71726 non-null  object
 4    service                     71726 non-null  object
 5    flow_duration               71726 non-null  float64
 6    fwd_pkts_tot                71726 non-null  int64
 7    bwd_pkts_tot                71726 non-null  int64
 8    fwd_data_pkts_tot           71726 non-null  int64
 9    bwd_data_pkts_tot           71726 non-null  int64
 10   fwd_pkts_per_sec            71726 non-null  float64
 11   bwd_pkts_per_sec            71726 non-null  float64
 12   flow_pkts_per_sec           71726 non-null  float64
 13   down_up_ratio               71726 non-null  float64
 14   fwd_header_size_tot         71726 non-null  int64
 15   fwd_header_size_min         71726 non-null  int64
 16   fwd_header_size_max         71726 non-null  int64
 17   bwd_header_size_tot         71726 non-null  int64
 18   bwd_header_size_min         71726 non-null  int64
 19   bwd_header_size_max         71726 non-null  int64
 20   flow_FIN_flag_count         71726 non-null  int64
 21   flow_SYN_flag_count         71726 non-null  int64
 22   flow_RST_flag_count         71726 non-null  int64
 23   fwd_PSH_flag_count          71726 non-null  int64
 24   bwd_PSH_flag_count          71726 non-null  int64
 25   flow_ACK_flag_count         71726 non-null  int64
 26   fwd_URG_flag_count          71726 non-null  int64
 27   bwd_URG_flag_count          71725 non-null  float64
 28   flow_CWR_flag_count         71725 non-null  float64
 29   flow_ECE_flag_count         71725 non-null  float64
 30   fwd_pkts_payload.min        71725 non-null  float64
 31   fwd_pkts_payload.max        71725 non-null  float64
 32   fwd_pkts_payload.tot        71725 non-null  float64
 33   fwd_pkts_payload.avg        71725 non-null  float64
 34   fwd_pkts_payload.std        71725 non-null  float64
 35   bwd_pkts_payload.min        71725 non-null  float64
 36   bwd_pkts_payload.max        71725 non-null  float64
 37   bwd_pkts_payload.tot        71725 non-null  float64
 38   bwd_pkts_payload.avg        71725 non-null  float64
 39   bwd_pkts_payload.std        71725 non-null  float64
 40   flow_pkts_payload.min       71725 non-null  float64
 41   flow_pkts_payload.max       71725 non-null  float64
 42   flow_pkts_payload.tot       71725 non-null  float64
 43   flow_pkts_payload.avg       71725 non-null  float64
 44   flow_pkts_payload.std       71725 non-null  float64
 45   fwd_iat.min                 71725 non-null  float64
 46   fwd_iat.max                 71725 non-null  float64
 47   fwd_iat.tot                 71725 non-null  float64
```

```
 48  fwd_iat.avg                71725 non-null  float64
 49  fwd_iat.std                71725 non-null  float64
 50  bwd_iat.min                71725 non-null  float64
 51  bwd_iat.max                71725 non-null  float64
 52  bwd_iat.tot                71725 non-null  float64
 53  bwd_iat.avg                71725 non-null  float64
 54  bwd_iat.std                71725 non-null  float64
 55  flow_iat.min               71725 non-null  float64
 56  flow_iat.max               71725 non-null  float64
 57  flow_iat.tot               71725 non-null  float64
 58  flow_iat.avg               71725 non-null  float64
 59  flow_iat.std               71725 non-null  float64
 60  payload_bytes_per_second  71725 non-null  float64
 61  fwd_subflow_pkts           71725 non-null  float64
 62  bwd_subflow_pkts           71725 non-null  float64
 63  fwd_subflow_bytes          71725 non-null  float64
 64  bwd_subflow_bytes          71725 non-null  float64
 65  fwd_bulk_bytes             71725 non-null  float64
 66  bwd_bulk_bytes             71725 non-null  float64
 67  fwd_bulk_packets           71725 non-null  float64
 68  bwd_bulk_packets           71725 non-null  float64
 69  fwd_bulk_rate              71725 non-null  float64
 70  bwd_bulk_rate              71725 non-null  float64
 71  active.min                 71725 non-null  float64
 72  active.max                 71725 non-null  float64
 73  active.tot                 71725 non-null  float64
 74  active.avg                 71725 non-null  float64
 75  active.std                 71725 non-null  float64
 76  idle.min                   71725 non-null  float64
 77  idle.max                   71725 non-null  float64
 78  idle.tot                   71725 non-null  float64
 79  idle.avg                   71725 non-null  float64
 80  idle.std                   71725 non-null  float64
 81  fwd_init_window_size       71725 non-null  float64
 82  bwd_init_window_size       71725 non-null  float64
 83  fwd_last_window_size       71725 non-null  float64
 84  Attack_type                71725 non-null  object
dtypes: float64(62), int64(20), object(3)
memory usage: 46.5+ MB
```

```
df.describe()
```

{"type":"dataframe"}

Transform

```
# create df copy
iot_22_df = df.copy()
```

1. Attack type distribution

```
attack_type_distribution =
pd.DataFrame(iot_22_df['Attack_type'].value_counts())
attack_type_distribution
```

{"summary":"{\n \"name\": \"attack_type_distribution\",\n \"rows\": 6,\n \"fields\": [\n {\n \"column\": \"Attack_type\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 6,\n \"samples\": [\n \"DOS_SYN_Hping\",\n \"Thing_Speak\",\n \"Wipro_bulb\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"count\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 19391,\n \"min\": 253,\n \"max\": 50934,\n \"num_unique_values\": 6,\n \"samples\": [\n 50934,\n 8108,\n 253\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"attack_type_distribution"}

```
# Attack_type percentage
attack_type_sum = attack_type_distribution['count'].sum()

# percentage computation, round off 2 decimal places
attack_type_distribution['percentage'] =
(attack_type_distribution['count'] / attack_type_sum) * 100
attack_type_distribution['percentage'] =
attack_type_distribution['percentage'].round(2)

attack_type_distribution
```

{"summary":"{\n \"name\": \"attack_type_distribution\",\n \"rows\": 6,\n \"fields\": [\n {\n \"column\": \"Attack_type\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 6,\n \"samples\": [\n \"DOS_SYN_Hping\",\n \"Thing_Speak\",\n \"Wipro_bulb\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"count\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 19391,\n \"min\": 253,\n \"max\": 50934,\n \"num_unique_values\": 6,\n \"samples\": [\n 50934,\n 8108,\n 253\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"percentage\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 27.035944037521606,\n \"min\": 0.35,\n \"max\": 71.01,\n \"num_unique_values\": 6,\n \"samples\": [\n 71.01,\n 11.3,\n 0.35\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"attack_type_distribution"}

2. Protocol vs Attack type, Service vs Attack type

```
# service column: Change category '-' with 'None'
iot_22_df['service'] = iot_22_df['service'].astype('category')
iot_22_df['service'] =
iot_22_df['service'].cat.rename_categories({'-':'None'})
iot_22_df['service']
```

```
0        mqtt
1        mqtt
2        mqtt
3        mqtt
4        mqtt
         ...
71721    None
71722    None
71723    None
71724    None
71725    None
Name: service, Length: 71726, dtype: category
Categories (8, object): ['None', 'dhcp', 'dns', 'http', 'irc', 'mqtt',
'ntp', 'ssl']
```

```
# protocol by attack type
proto_by_attack_type = iot_22_df.groupby('Attack_type')
['proto'].value_counts().unstack().fillna(0)
proto_by_attack_type
```


{"summary":"{\n \"name\": \"proto_by_attack_type\",\n \"rows\": 6,\n
\"fields\": [\n {\n \"column\": \"Attack_type\",\n
\"properties\": {\n \"dtype\": \"string\",\n
\"num_unique_values\": 6,\n \"samples\": [\n
\"ARP_poisioning\",\n \"DDOS_Slowloris\",\n
\"Wipro_bulb\"\n],\n \"semantic_type\": \"\",\n
\"description\": \"\"\n }\n },\n {\n \"column\":
\"icmp\",\n \"properties\": {\n \"dtype\": \"number\",\n
\"std\": 17.82601095777366,\n \"min\": 0.0,\n \"max\":
45.0,\n \"num_unique_values\": 4,\n \"samples\": [\n
0.0,\n 2.0,\n 8.0\n],\n
\"semantic_type\": \"\",\n \"description\": \"\"\n }\
n },\n {\n \"column\": \"tcp\",\n \"properties\": {\n
\"dtype\": \"number\",\n \"std\": 19997.276342042183,\n
\"min\": 182.0,\n \"max\": 50934.0,\n
\"num_unique_values\": 6,\n \"samples\": [\n 1927.0,\n
529.0,\n 182.0\n],\n \"semantic_type\": \"\",\
n \"description\": \"\"\n }\n },\n {\n
\"column\": \"udp\",\n \"properties\": {\n \"dtype\":
\"number\",\n \"std\": 2642.852455208198,\n \"min\":
0.0,\n \"max\": 5815.0,\n \"num_unique_values\": 5,\n
\"samples\": [\n 5.0,\n 69.0,\n 0.0\n
],\n \"semantic_type\": \"\",\n \"description\": \"\"\n

}\n   }\n]\
n}","type":"dataframe","variable_name":"proto_by_attack_type"}

```
# service by attack type
service_by_attack_type = iot_22_df.groupby('Attack_type')
['service'].value_counts().unstack().fillna(0)
service_by_attack_type
```

{"summary":"{\n \"name\": \"service_by_attack_type\",\n \"rows\":
6,\n \"fields\": [\n   {\n   \"column\": \"Attack_type\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 6,\n   \"samples\": [\n
\"ARP_poisioning\",\n   \"DDOS_Slowloris\",\n
\"Wipro_bulb\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"None\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 20732,\n   \"min\": 6,\n   \"max\": 50934,\n
\"num_unique_values\": 6,\n   \"samples\": [\n   546,\n
6,\n   45\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"dhcp\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 10,\n   \"min\": 0,\n   \"max\": 26,\n
\"num_unique_values\": 5,\n   \"samples\": [\n   2,\n
5,\n   0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"dns\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 2530,\n   \"min\": 0,\n   \"max\": 5583,\n
\"num_unique_values\": 5,\n   \"samples\": [\n   3,\n
53,\n   0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"http\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 1054,\n   \"min\": 0,\n   \"max\": 2667,\n
\"num_unique_values\": 5,\n   \"samples\": [\n   523,\n
2667,\n   0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"irc\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 17,\n   \"min\": 0,\n   \"max\": 43,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   43,\n
0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"mqtt\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 1686,\n   \"min\": 0,\n   \"max\": 4132,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   4132,\n
0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"ntp\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 44,\n   \"min\": 0,\n   \"max\": 110,\n
\"num_unique_values\": 3,\n   \"samples\": [\n   7,\n
0\n   ],\n   \"semantic_type\": \"\",\n

\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"ssl\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 657,\n   \"min\": 0,\n   \"max\": 1459,\n   \"num_unique_values\": 4,\n   \"samples\": [\n   0,\n   107\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"service_by_attack_type"}

1. Flow duration stats per Attack type

```
# flow stats by atk type
flow_stats_by_atk = iot_22_df.groupby('Attack_type').agg({
    'flow_duration': ['mean', 'std']
})

flow_stats_by_atk
```

{"summary":"{\n \"name\": \"flow_stats_by_atk\",\n \"rows\": 6,\n \"fields\": [\n   {\n   \"column\": [\n   \"Attack_type\",\n   \"\"\n   ],\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 6,\n   \"samples\": [\n   \"ARP_poisioning\",\n   \"DDOS_Slowloris\",\n   \"Wipro_bulb\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": [\n   \"flow_duration\",\n   \"mean\"\n   ],\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 233.98754266161103,\n   \"min\": 3.0095417599246086e-06,\n   \"max\": 586.8457270158103,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   15.893538231612904,\n   14.69914792883895,\n   586.8457270158103\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": [\n   \"flow_duration\",\n   \"std\"\n   ],\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 1106.4548722268719,\n   \"min\": 1.9709141124334797e-06,\n   \"max\": 2738.8916374814985,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   108.26107023602722,\n   14.124797178358959,\n   2738.8916374814985\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"flow_stats_by_atk"}

1. Packets per second and bytes per second vs Attack type

```
# correlation of attack classes to fwd_pkts_per_sec, bwd_pkts_per_sec, payload_bytes_per_second
atk_classes_rate_corr = iot_22_df.groupby('Attack_type')
[['fwd_pkts_per_sec', 'bwd_pkts_per_sec',
'payload_bytes_per_second']].corr().unstack().fillna(0)

atk_classes_rate_corr
```

{"summary":"{\n \"name\": \"atk_classes_rate_corr\",\n \"rows\": 6,\n \"fields\": [\n {\n \"column\": [\n \"Attack_type\",\n \"\"\n],\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 6,\n \"samples\": [\n \"ARP_poisioning\",\n \"DDOS_Slowloris\",\n \"Wipro_bulb\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"fwd_pkts_per_sec\",\n \"fwd_pkts_per_sec\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.0,\n \"min\": 1.0,\n \"max\": 1.0,\n \"num_unique_values\": 1,\n \"samples\": [\n 1.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"fwd_pkts_per_sec\",\n \"bwd_pkts_per_sec\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.5023464736664688,\n \"min\": -0.18484697599173242,\n \"max\": 1.0,\n \"num_unique_values\": 6,\n \"samples\": [\n 0.48129456575804463\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"fwd_pkts_per_sec\",\n \"payload_bytes_per_second\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.37877644578670977,\n \"min\": 0.03697781285828206,\n \"max\": 1.0000000000000078,\n \"num_unique_values\": 6,\n \"samples\": [\n 0.42887979285454375\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"bwd_pkts_per_sec\",\n \"fwd_pkts_per_sec\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.5023464736664688,\n \"min\": -0.18484697599173242,\n \"max\": 1.0,\n \"num_unique_values\": 6,\n \"samples\": [\n 0.48129456575804463\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"bwd_pkts_per_sec\",\n \"bwd_pkts_per_sec\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.0,\n \"min\": 1.0,\n \"max\": 1.0,\n \"num_unique_values\": 1,\n \"samples\": [\n 1.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"bwd_pkts_per_sec\",\n \"payload_bytes_per_second\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.4325289999009616,\n \"min\": -0.1422088700024508,\n \"max\": 1.0000000000000078,\n \"num_unique_values\": 6,\n \"samples\": [\n 0.893117541237937\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"payload_bytes_per_second\",\n \"fwd_pkts_per_sec\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.37877644578670977,\n \"min\": 0.03697781285828206,\n \"max\": 1.0000000000000078,\n \"num_unique_values\": 6,\n \"samples\": [\n 0.42887979285454375\n],\n

```
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": [\n
\"payload_bytes_per_second\",\n        \"bwd_pkts_per_sec\"\n      ],\
n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.4325289999009616,\n        \"min\": -0.1422088700024508,\n
\"max\": 1.0000000000000078,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          0.893117541237937\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": [\n
\"payload_bytes_per_second\",\n        \"payload_bytes_per_second\"\n
],\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.0,\n        \"min\": 1.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          1.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\
n}","type":"dataframe","variable_name":"atk_classes_rate_corr"}
```


1. Time-based features vs Attack types

```
time_features_atk_corr = iot_22_df.groupby('Attack_type')
[['fwd_iat.avg', 'bwd_iat.avg']].corr().fillna(0)

time_features_atk_corr
```


```
{"summary":"{\n  \"name\": \"time_features_atk_corr\",\n  \"rows\":
10,\n  \"fields\": [\n    {\n      \"column\": \"fwd_iat.avg\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.43359189582065144,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 6,\n        \"samples\": [\n          1.0,\n
0.8409532095740062,\n          0.07788598351306474\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"bwd_iat.avg\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.43359189582065144,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 6,\n        \"samples\": [\n
0.8409532095740062,\n          1.0,\n          0.07788598351306474\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\
n}","type":"dataframe","variable_name":"time_features_atk_corr"}
```


1. Network flag counts per Attack type

```
# network flag counts on intrusion patterns
atk_classes_flag_counts = iot_22_df.groupby('Attack_type').agg({
    'flow_SYN_flag_count':'count',
    'flow_RST_flag_count':'count',
    'fwd_PSH_flag_count':'count'
})

atk_classes_flag_counts
```

{"summary":"{\n \"name\": \"atk_classes_flag_counts\",\n \"rows\": 6,\n \"fields\": [\n {\n \"column\": \"Attack_type\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 6,\n \"samples\": [\n \"ARP_poisioning\",\n \"DDOS_Slowloris\",\n \"Wipro_bulb\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"flow_SYN_flag_count\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 19391,\n \"min\": 253,\n \"max\": 50934,\n \"num_unique_values\": 6,\n \"samples\": [\n 7750,\n 534,\n 253\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"flow_RST_flag_count\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 19391,\n \"min\": 253,\n \"max\": 50934,\n \"num_unique_values\": 6,\n \"samples\": [\n 7750,\n 534,\n 253\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"fwd_PSH_flag_count\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 19391,\n \"min\": 253,\n \"max\": 50934,\n \"num_unique_values\": 6,\n \"samples\": [\n 7750,\n 534,\n 253\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"atk_classes_flag_counts"}

Load

1. What is the distribution of the Attack_type classes (normal vs. various attacks), and what percentage of the 123,117 instances does each class comprise?

```
attack_type_distribution
```

{"summary":"{\n \"name\": \"attack_type_distribution\",\n \"rows\": 6,\n \"fields\": [\n {\n \"column\": \"Attack_type\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 6,\n \"samples\": [\n \"DOS_SYN_Hping\",\n \"Thing_Speak\",\n \"Wipro_bulb\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"count\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 19391,\n \"min\": 253,\n \"max\": 50934,\n \"num_unique_values\": 6,\n \"samples\": [\n 50934,\n 8108,\n 253\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"percentage\",\n \"properties\": {\n \"dtype\":

\"number\",\n   \"std\": 27.035944037521606,\n   \"min\": 0.35,\n   \"max\": 71.01,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   71.01,\n   11.3,\n   0.35\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"attack_type_distribution"}

```
# Attack types below 1%
low_percentage_attacks =
attack_type_distribution[attack_type_distribution['percentage'] <
1].index
iot_22_df['Attack_type'] =
iot_22_df['Attack_type'].replace(low_percentage_attacks, 'Others')

# Crate new df with others category
attack_type_distribution =
pd.DataFrame(iot_22_df['Attack_type'].value_counts())

# recalculate percentages
attack_type_sum = attack_type_distribution['count'].sum()
attack_type_distribution['percentage'] =
(attack_type_distribution['count'] / attack_type_sum) * 100
attack_type_distribution['percentage'] =
attack_type_distribution['percentage'].round(2)

attack_type_distribution
```

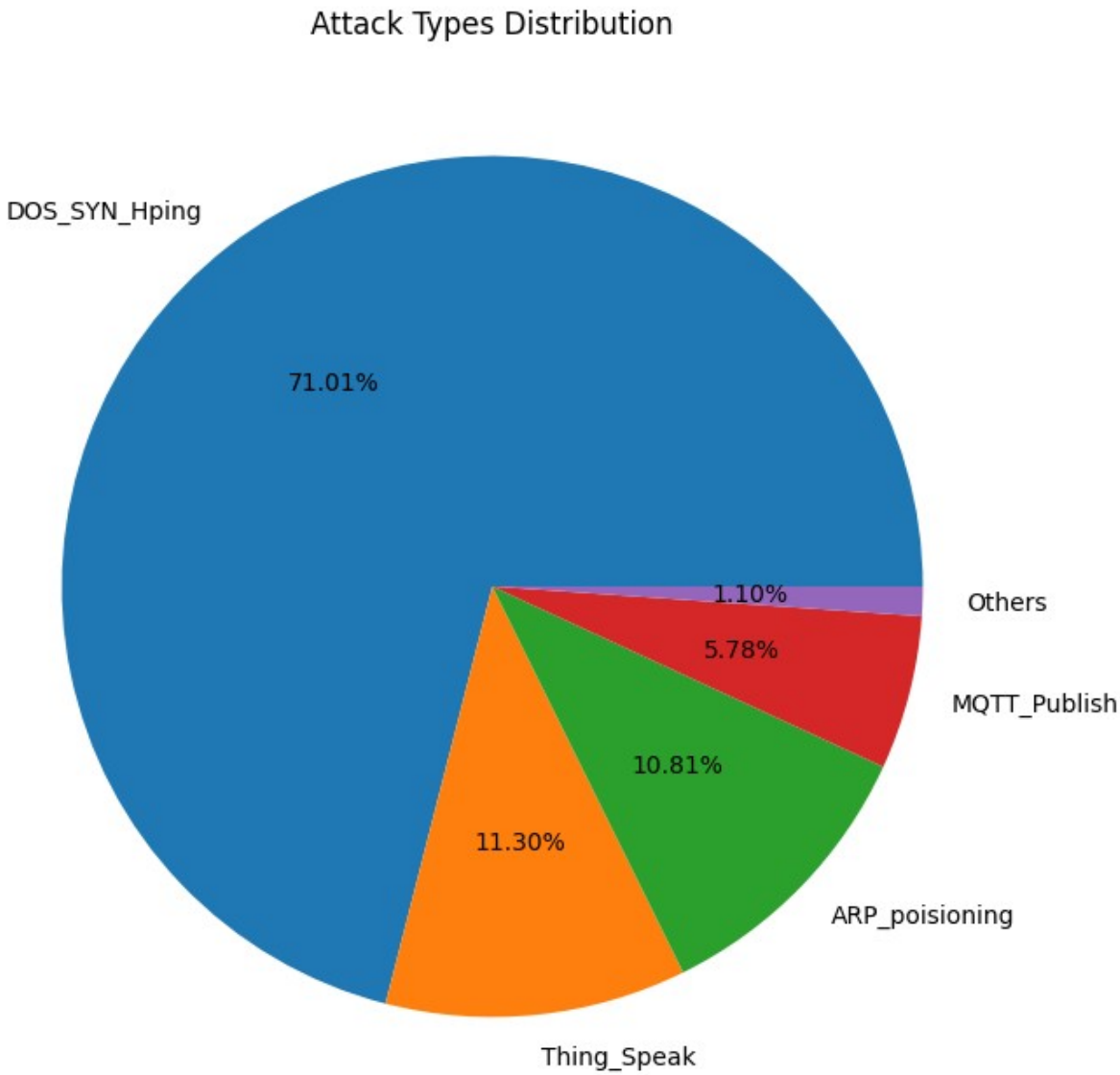
{"summary":"{\n \"name\": \"attack_type_distribution\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"Attack_type\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"Thing_Speak\",\n \"Others\",\n \"ARP_poisioning\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"count\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 20669,\n \"min\": 787,\n \"max\": 50934,\n \"num_unique_values\": 5,\n \"samples\": [\n 8108,\n 787,\n 7750\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"percentage\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 28.81659504521657,\n \"min\": 1.1,\n \"max\": 71.01,\n \"num_unique_values\": 5,\n \"samples\": [\n 11.3,\n 1.1,\n 10.81\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"attack_type_distribution"}

```
labels = attack_type_distribution.index

fig, ax = plt.subplots(figsize=(10,8))
```

```
ax.pie(attack_type_distribution['percentage'], labels=labels,
autopct='%1.2f%%', labeldistance=1.1)
plt.title("Attack Types Distribution")

plt.show()
```



2. How do the categorical features proto (protocol) and service vary across different attack types and normal traffic patterns?

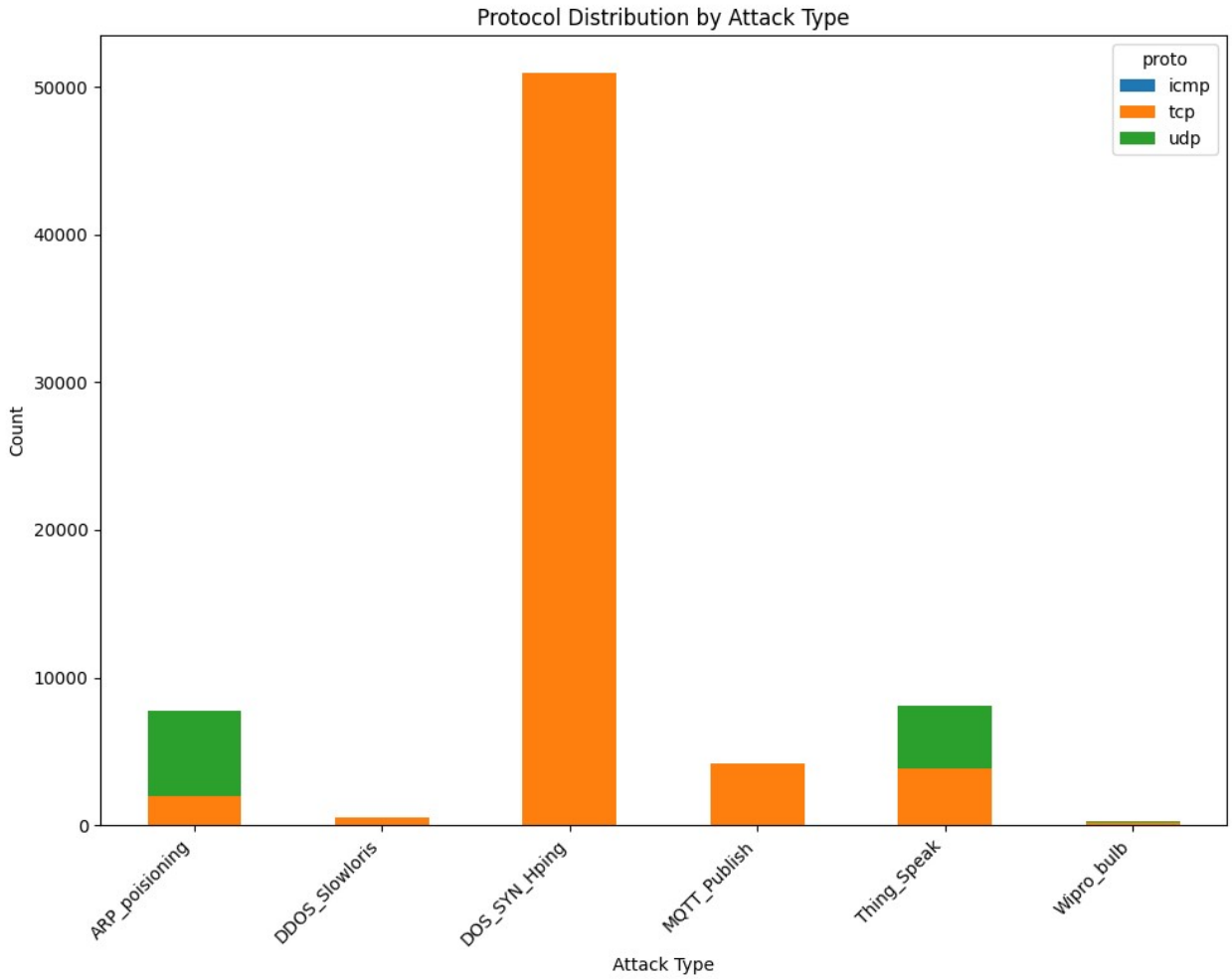
```
proto_by_attack_type
```


```
{"summary":"{\n  \"name\": \"proto_by_attack_type\",\n  \"rows\": 6,\n
\"fields\": [\n    {\n      \"column\": \"Attack_type\",\n
```

\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 6,\n   \"samples\": [\n
\"ARP_poisioning\",\n   \"DDOS_Slowloris\",\n
\"Wipro_bulb\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"icmp\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 17.82601095777366,\n   \"min\": 0.0,\n   \"max\":
45.0,\n   \"num_unique_values\": 4,\n   \"samples\": [\n
0.0,\n   2.0,\n   8.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"tcp\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 19997.276342042183,\n
\"min\": 182.0,\n   \"max\": 50934.0,\n
\"num_unique_values\": 6,\n   \"samples\": [\n   1927.0,\n
529.0,\n   182.0\n   ],\n   \"semantic_type\": \"\",\
n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"udp\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 2642.852455208198,\n   \"min\":
0.0,\n   \"max\": 5815.0,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   5.0,\n   69.0,\n   0.0\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   }\n]\
n}","type":"dataframe","variable_name":"proto_by_attack_type"}

```
# Protocol by Attack type Visualization
plt.figure(figsize=(10,8))
proto_by_attack_type.plot(kind='bar', stacked=True, ax=plt.gca())
plt.title('Protocol Distribution by Attack Type')
plt.xlabel('Attack Type')
plt.ylabel('Count')
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



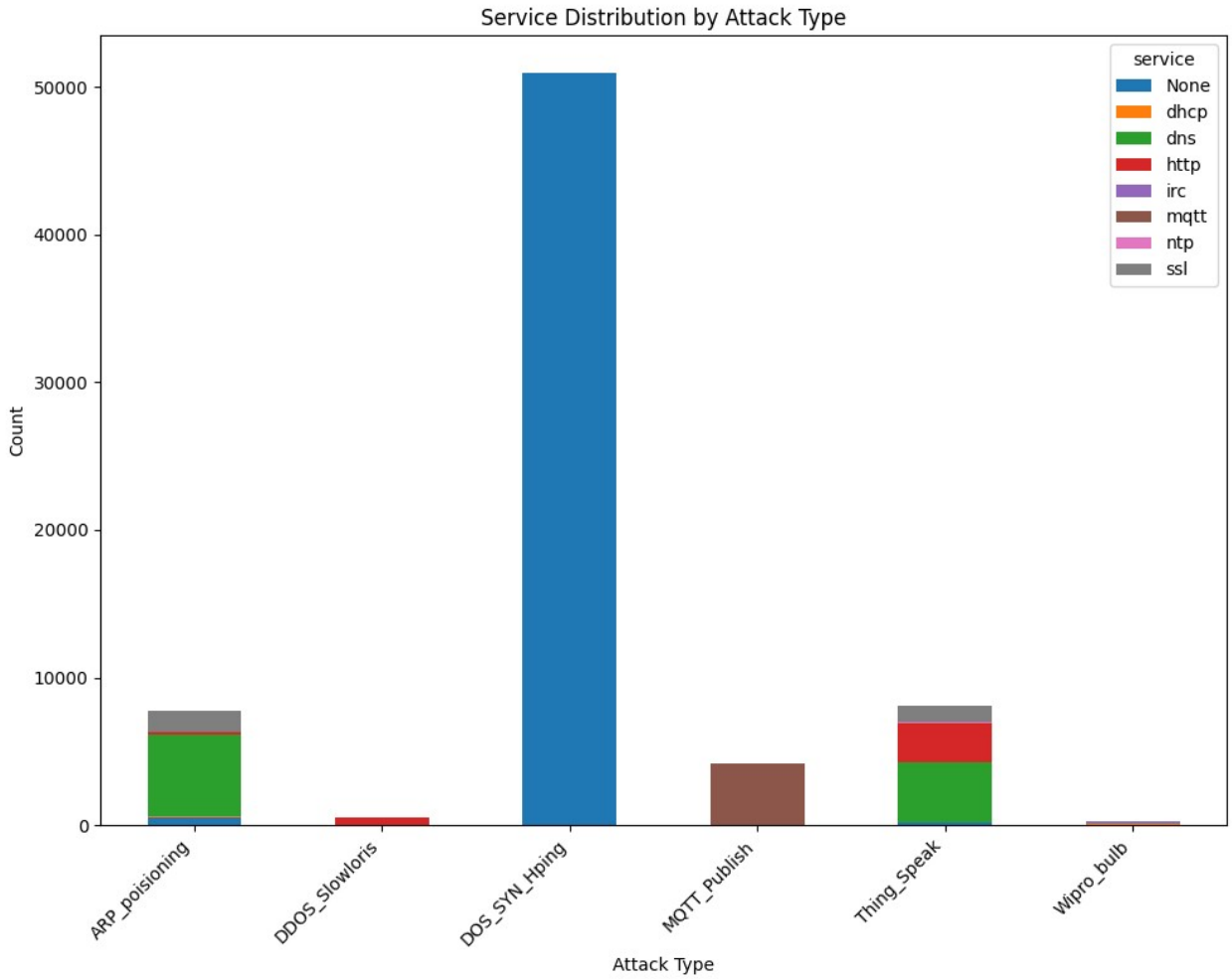
service_by_attack_type

{"summary":"{\n \"name\": \"service_by_attack_type\",\n \"rows\": 6,\n \"fields\": [\n {\n \"column\": \"Attack_type\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 6,\n \"samples\": [\n \"ARP_poisioning\",\n \"DDOS_Slowloris\",\n \"Wipro_bulb\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"None\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 20732,\n \"min\": 6,\n \"max\": 50934,\n \"num_unique_values\": 6,\n \"samples\": [\n 546,\n 6,\n 45\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"dhcp\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 10,\n \"min\": 0,\n \"max\": 26,\n \"num_unique_values\": 5,\n \"samples\": [\n 2,\n 5,\n 0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"dns\",\n \"properties\": {\n \"dtype\": \"number\",\n

\"std\": 2530,\n   \"min\": 0,\n   \"max\": 5583,\n
\"num_unique_values\": 5,\n   \"samples\": [\n   3,\n
53,\n   0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"http\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 1054,\n   \"min\": 0,\n   \"max\": 2667,\n
\"num_unique_values\": 5,\n   \"samples\": [\n   523,\n
2667,\n   0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"irc\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 17,\n   \"min\": 0,\n   \"max\": 43,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   43,\n
0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"mqtt\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 1686,\n   \"min\": 0,\n   \"max\": 4132,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   4132,\n
0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"ntp\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 44,\n   \"min\": 0,\n   \"max\": 110,\n
\"num_unique_values\": 3,\n   \"samples\": [\n   7,\n
0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"ssl\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 657,\n   \"min\": 0,\n   \"max\": 1459,\n
\"num_unique_values\": 4,\n   \"samples\": [\n   0,\n
107\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"service_by_attack_type"}

```
# Service by Attack type Visualization
plt.figure(figsize=(10,8))
service_by_attack_type.plot(kind='bar', stacked=True, ax=plt.gca())
plt.title('Service Distribution by Attack Type')
plt.xlabel('Attack Type')
plt.ylabel('Count')
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



3. What are the mean and standard deviation of flow_duration for each Attack_type, and are differences statistically significant?

```
flow_stats_by_atk
```


```
{"summary":"{\n  \"name\": \"flow_stats_by_atk\",\n  \"rows\": 6,\n  \"fields\": [\n  {\n  \"column\": [\n  \"Attack_type\",\n  \"\"\n  ],\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 6,\n  \"samples\": [\n  \"ARP_poisioning\",\n  \"DDOS_Slowloris\",\n  \"Wipro_bulb\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": [\n  \"flow_duration\",\n  \"mean\"\n  ],\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 233.98754266161103,\n  \"min\": 3.0095417599246086e-06,\n  \"max\": 586.8457270158103,\n  \"num_unique_values\": 6,\n  \"samples\": [\n  15.893538231612904,\n  14.69914792883895,\n  586.8457270158103\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": [\n  \"flow_duration\",\n
```

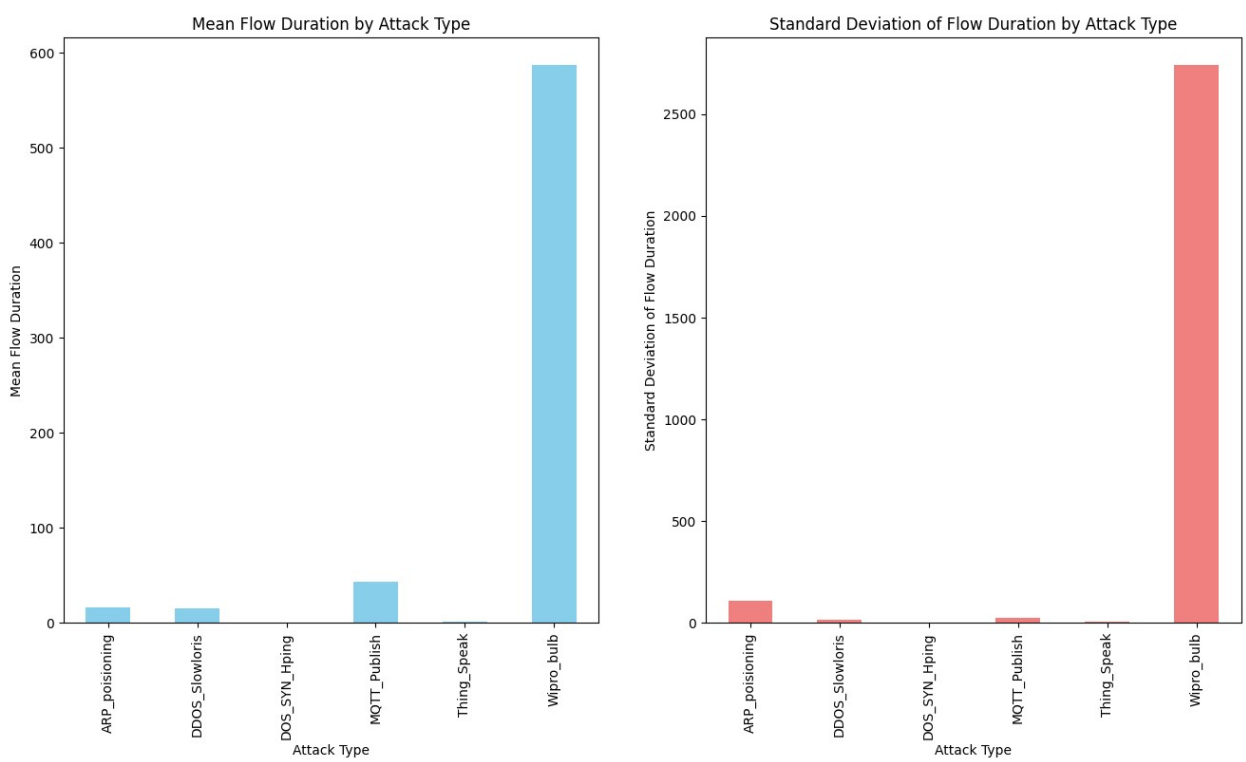
\"std\"\n   ],\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 1106.4548722268719,\n   \"min\": 1.9709141124334797e-06,\n   \"max\": 2738.8916374814985,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   108.26107023602722,\n   14.124797178358959,\n   2738.8916374814985\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"flow_stats_by_atk"}

```
fig, axes = plt.subplots(1, 2, figsize=(16, 8))

# mean flow duration
flow_stats_by_atk['flow_duration']['mean'].plot(kind='bar',
ax=axes[0], color='skyblue')
axes[0].set_title('Mean Flow Duration by Attack Type')
axes[0].set_xlabel('Attack Type')
axes[0].set_ylabel('Mean Flow Duration')

# standard deviation of flow duration
flow_stats_by_atk['flow_duration']['std'].plot(kind='bar', ax=axes[1],
color='lightcoral')
axes[1].set_title('Standard Deviation of Flow Duration by Attack
Type')
axes[1].set_xlabel('Attack Type')
axes[1].set_ylabel('Standard Deviation of Flow Duration')

plt.show()
```



4. Which continuous features (e.g., fwd_pkts_per_sec, bwd_pkts_per_sec, payload_bytes_per_second) exhibit the highest correlation with specific attack classes?

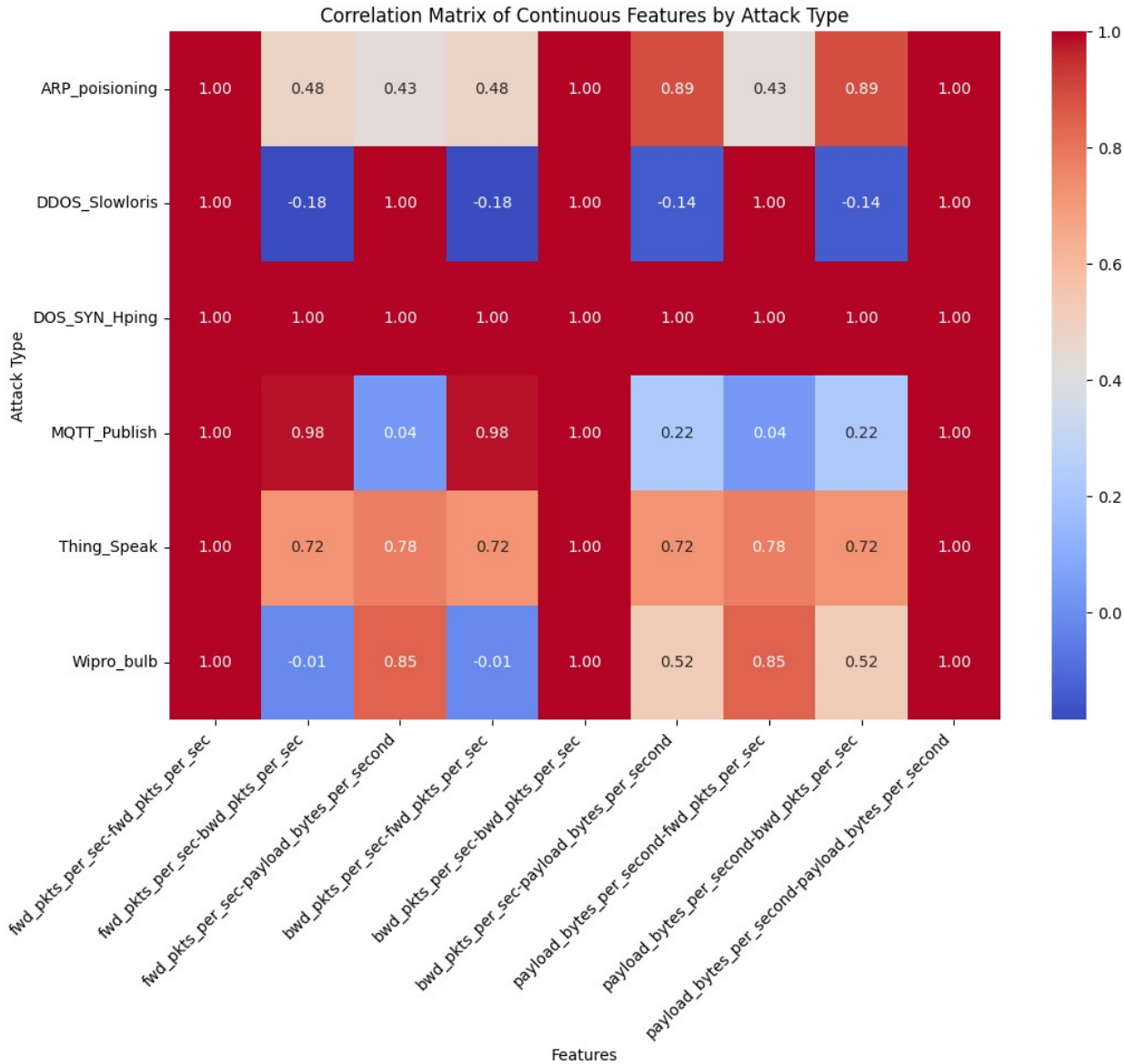
```
atk_classes_rate_corr
```


```
{"summary":"{\n  \"name\": \"atk_classes_rate_corr\",\n  \"rows\": 6,\
n  \"fields\": [\n    {\n      \"column\": [\n
\"Attack_type\",\n        \"\"\n      ],\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 6,\n
\"samples\": [\n          \"ARP_poisioning\",\n
\"DDOS_Slowloris\",\n          \"Wipro_bulb\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": [\n        \"fwd_pkts_per_sec\",\n
\"fwd_pkts_per_sec\"\n      ],\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.0,\n        \"min\": 1.0,\n
\"max\": 1.0,\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          1.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": [\n
\"fwd_pkts_per_sec\",\n        \"bwd_pkts_per_sec\"\n      ],\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.5023464736664688,\n        \"min\": -0.18484697599173242,\n
\"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\":
[\n          0.48129456575804463\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": [\n        \"fwd_pkts_per_sec\",\n
\"payload_bytes_per_second\"\n      ],\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.37877644578670977,\n
\"min\": 0.03697781285828206,\n        \"max\": 1.0000000000000078,\n
\"num_unique_values\": 6,\n        \"samples\": [\n
0.42887979285454375\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": [\n
\"bwd_pkts_per_sec\",\n        \"fwd_pkts_per_sec\"\n      ],\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.5023464736664688,\n        \"min\": -0.18484697599173242,\n
\"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\":
[\n          0.48129456575804463\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": [\n        \"bwd_pkts_per_sec\",\n
\"bwd_pkts_per_sec\"\n      ],\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.0,\n        \"min\": 1.0,\n
\"max\": 1.0,\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          1.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": [\n
\"bwd_pkts_per_sec\",\n        \"payload_bytes_per_second\"\n      ],\
n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.4325289999009616,\n        \"min\": -0.1422088700024508,\n
\"max\": 1.0000000000000078,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          0.893117541237937\n        ],\n
```

\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": [\n
\"payload_bytes_per_second\",\n   \"fwd_pkts_per_sec\"\n   ],\
n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0.37877644578670977,\n   \"min\": 0.03697781285828206,\n
\"max\": 1.0000000000000078,\n   \"num_unique_values\": 6,\n
\"samples\": [\n   0.42887979285454375\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": [\n
\"payload_bytes_per_second\",\n   \"bwd_pkts_per_sec\"\n   ],\
n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0.4325289999009616,\n   \"min\": -0.1422088700024508,\n
\"max\": 1.0000000000000078,\n   \"num_unique_values\": 6,\n
\"samples\": [\n   0.893117541237937\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": [\n
\"payload_bytes_per_second\",\n   \"payload_bytes_per_second\"\n
],\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0.0,\n   \"min\": 1.0,\n   \"max\": 1.0,\n
\"num_unique_values\": 1,\n   \"samples\": [\n   1.0\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   }\n]\
n}","type":"dataframe","variable_name":"atk_classes_rate_corr"}

```
plt.figure(figsize=(12, 8))
sns.heatmap(atk_classes_rate_corr, annot=True, cmap='coolwarm',
fmt=".2f")
plt.title('Correlation Matrix of Continuous Features by Attack Type')
plt.xlabel('Features')
plt.ylabel('Attack Type')
plt.xticks(rotation=45, ha='right')

plt.show()
```



5. How do time-based features like fwd_iat.avg and bwd_iat.avg (mean inter-arrival times) differ between various attack types and normal traffic?

```
time_features_atk_corr
```

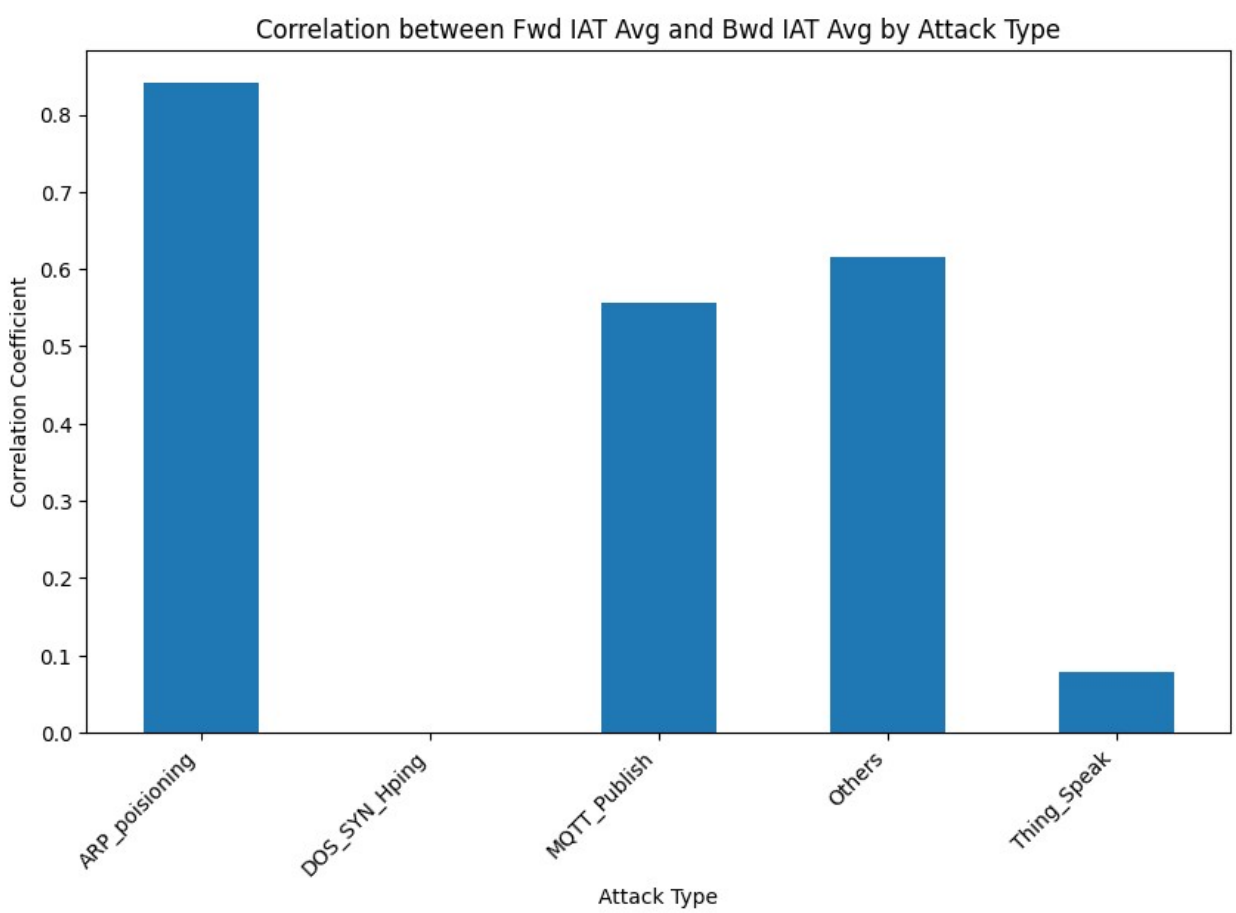

```
{"summary":"{\n  \"name\": \"time_features_atk_corr\",\n  \"rows\": 10,\n  \"fields\": [\n  {\n  \"column\": \"fwd_iat.avg\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 0.43359189582065144,\n  \"min\": 0.0,\n  \"max\": 1.0,\n  \"num_unique_values\": 6,\n  \"samples\": [\n  1.0,\n  0.8409532095740062,\n  0.07788598351306474\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"bwd_iat.avg\",\n
```

\"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.43359189582065144,\n \"min\": 0.0,\n \"max\": 1.0,\n \"num_unique_values\": 6,\n \"samples\": [\n 0.8409532095740062,\n 1.0,\n 0.07788598351306474\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"time_features_atk_corr"}

```
cross_correlation = time_features_atk_corr.loc[(slice(None), 'fwd_iat.avg'), 'bwd_iat.avg']
cross_correlation = cross_correlation.reset_index(level=1, drop=True)

plt.figure(figsize=(10, 6))
cross_correlation.plot(kind='bar')
plt.title('Correlation between Fwd IAT Avg and Bwd IAT Avg by Attack Type')
plt.xlabel('Attack Type')
plt.ylabel('Correlation Coefficient')
plt.xticks(rotation=45, ha='right')


plt.show()
```



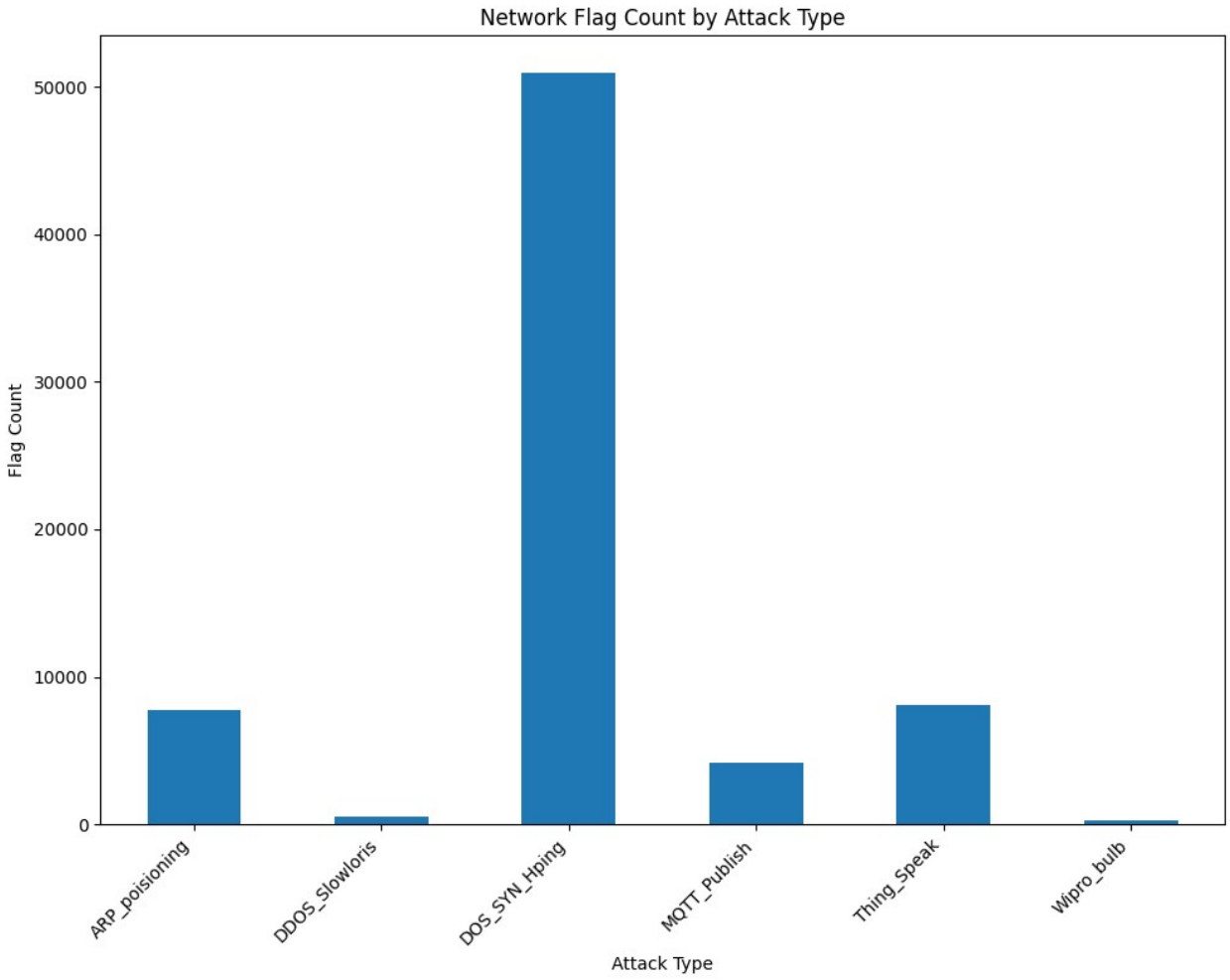
6. Which network flag counts (e.g., flow_SYN_flag_count, flow_RST_flag_count, fwd_PSH_flag_count) are most indicative of specific intrusion patterns?

```
atk_classes_flag_counts
```


```
{"summary":"{\n  \"name\": \"atk_classes_flag_counts\",\n  \"rows\": 6,\n  \"fields\": [\n  {\n  \"column\": \"Attack_type\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 6,\n  \"samples\": [\n  \"ARP_poisioning\",\n  \"DDOS_Slowloris\",\n  \"Wipro_bulb\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"flow_SYN_flag_count\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 19391,\n  \"min\": 253,\n  \"max\": 50934,\n  \"num_unique_values\": 6,\n  \"samples\": [\n  7750,\n  534,\n  253\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"flow_RST_flag_count\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 19391,\n  \"min\": 253,\n  \"max\": 50934,\n  \"num_unique_values\": 6,\n  \"samples\": [\n  7750,\n  534,\n  253\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"fwd_PSH_flag_count\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 19391,\n  \"min\": 253,\n  \"max\": 50934,\n  \"num_unique_values\": 6,\n  \"samples\": [\n  7750,\n  534,\n  253\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe","variable_name":"atk_classes_flag_counts"}
```


```
plt.figure(figsize=(10,8))
atk_classes_flag_counts['flow_SYN_flag_count'].plot(kind='bar')
plt.title("Network Flag Count by Attack Type")
plt.xlabel("Attack Type")
plt.ylabel("Flag Count")
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```


Network Flag Count by Attack Type
Flag Count
50000
40000
30000
20000
10000
0
ARP_poisioning
DDOS_Slowloris
DOS_SYN_Hping
MQTT_Publish
Thing_Speak
Wipro_bulb
Attack Type